Authorship Memo — Iteration 1

To: The engineer who inherits this repository
From: Reggie Johnson
Project: Content Creation Project (RJ-Capstone)
Date: 31 October 2026
Tag: iteration-1

1. The most assistant-shaped file

The single most assistant-shaped file in this repository is **src/app/render.js**. It is not the hardest file intellectually — that is closer to **src/app/project-service.js**, where the acceptance criteria live as explicit error codes — but it is the file where the largest share of what shipped arrived as drafted structure. Roughly **sixty to seventy percent** of the first draft of render.js came from an assistant session: the DOM construction pattern, the form markup for "Add section," the status `select` loop, and the draft panel layout. What survived into the tag is closer to **half of the line count as shaped by me**, because I tore out auto-save-on-every-keystroke, forced a dirty flag, restored caret position after re-render, and made every visible string match the requirement vocabulary instead of generic UI copy.

I am naming render.js on purpose. Naming the CI workflow or a README would be an evasion. The grader can open render.js and see the assistant's fingerprints in the scaffolding. Owning that is part of the job.

2. Defending it as my work — and where I still owe work

I treat render.js as my work because the **behavior contract** is mine, even where the DOM boilerplate was not. The design decisions that matter for Iteration 1 are:

• **No silent auto-save.** The assistant's first sketch saved on every input event. That would have made the dirty-state banner meaningless and would have written partial keystrokes into localStorage on every character. I rejected that. Save is explicit (FR-SAVE-09). The dirty flag drives the "Save*" label and blocks section switches with `UNSAVED_CHANGES`.

• **Error codes match the issues list.** Empty add → `EMPTY_SECTION_NAME`. Unknown section → `UNKNOWN_SECTION`. Bad status → `INVALID_STATUS`. Those strings are not decorative; they are the acceptance criteria from docs/issues/issues.md, surfaced in the UI banner so a stranger can see the failure path without opening DevTools.

• **Single status vocabulary.** Status options are not hard-coded twice. render.js imports `VALID_STATUSES` from project-service.js. That decision was mine after the first draft duplicated the list.

The tests that prove the requirement side of this work live mainly next to the domain layer, not inside render.js: `tests/unit/project-service.test.js` covers FR-MAP-01, FR-DRAFT-02, and FR-PROG-03, including the three failure codes above. Store load/save/seed is covered in `project-store.test.js`. Smoke still asserts the seeded "Introduction" row from Persistence. I can explain every branch in project-service.js without re-reading it. I cannot yet say the same about every caret-restoration detail in the re-render path of main.js; that is a gap I am

calling out, not hiding. Before Week 16 I will either simplify the paint cycle so caret hacks are unnecessary, or write a focused note in the code that states the invariant in one sentence I can defend under pressure.

3. The 2 a.m. question

If render.js (or the panel it builds) failed in “production” at 2 a.m. — meaning a user opens the GitHub Pages build and the section list is blank, or Save does nothing, or status never sticks — this is exactly what I do first, in order:

1. Open the browser console and localStorage. Check key `ccp:project:v1`. If the key is missing or holds non-JSON, that is a Persistence problem, not a paint problem. Run `JSON.parse(localStorage.getItem('ccp:project:v1'))` by hand.

2. If the document parses but the UI is empty, check whether `#app` exists and whether main.js threw before paint. The boot path catches and writes `Error: …` into the root — that message is the next clue.

3. If Save appears to work but reload loses data, I verify `ProjectStore.save` was actually called (banner must show a timestamp) and that the adapter is localStorage in the browser, not a leftover mental model from the MemoryAdapter used in Node tests.

4. Only after that do I touch render.js. Most “UI bugs” in this app so far have been data-shape bugs (missing `body` on old skeleton docs, which seed.js now backfills). I will not start by rewriting the form markup at 2 a.m.

4. One call I cannot delegate

The place I accepted output I could not fully explain on first read was the **full re-paint on every textarea input** combined with caret restoration in main.js. The assistant suggested re-rendering the whole app on each keystroke so the dirty flag and banner stayed in sync. That works, but it is not a design I would have chosen from a blank file, and the selection restore is a patch on top of a patch.

What I chose: **keep it for Iteration 1, with eyes open, and schedule a rewrite**. I did not silently pretend I invented it. Explaining it properly now: the paint function closes over handlers; each input updates the in-memory project, sets dirty, and calls paint(); because paint replaces the textarea node, the browser drops the caret; so after paint I re-query `#section-body` and set selection to the end. Cost so far: about **half an hour** to understand and stabilize, plus this memo. Cost still owed: a Week 11 change to update the textarea value in place (or debounce a partial render) so we do not rebuild the section list on every character. That is on the board as technical debt tied to FR-DRAFT-02 quality, not as a new feature.

5. Whose work is this?

This is **my work**, with assistant labor inside it — the same way a codebase can contain a library I did not author and still be my product. What makes that true is not a slogan about “using AI responsibly.” It is three concrete claims I am willing to be graded on:

First, **the requirements and the failure vocabulary are mine**. FR-MAP-01, FR-DRAFT-02, FR-PROG-03, and the error codes in the UI came from the issues list and the acceptance criteria I

committed to, not from a generic CRUD template.

Second, **I can operate the system when it breaks**. The 2 a.m. sequence above is not theater; it is the order I already used when seed shape and skeleton docs disagreed during Iteration 1.

Third, **I recorded what I did not write**. docs/ai-usage.md for 28–31 October states what was drafted, what I rejected, and how I verified. A repository that hides the assistant is not more mine; it is less inspectable. Ownership here means I can point at a line, say whether I typed it, whether I kept it, and what test or demo step proves it belongs.

If those three claims fail for a file, that file is not yet mine — and I have said so for the caret path. The rest of Iteration 1 I will defend.

Reggie Johnson
Content Creation Project — tag iteration-1